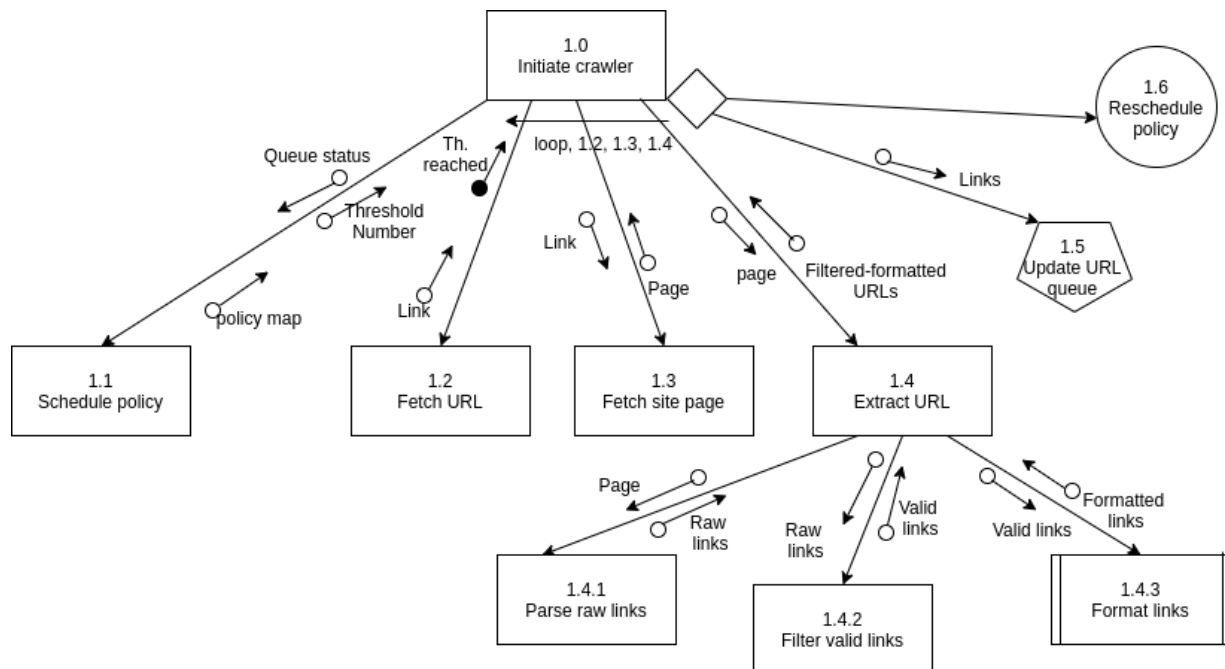
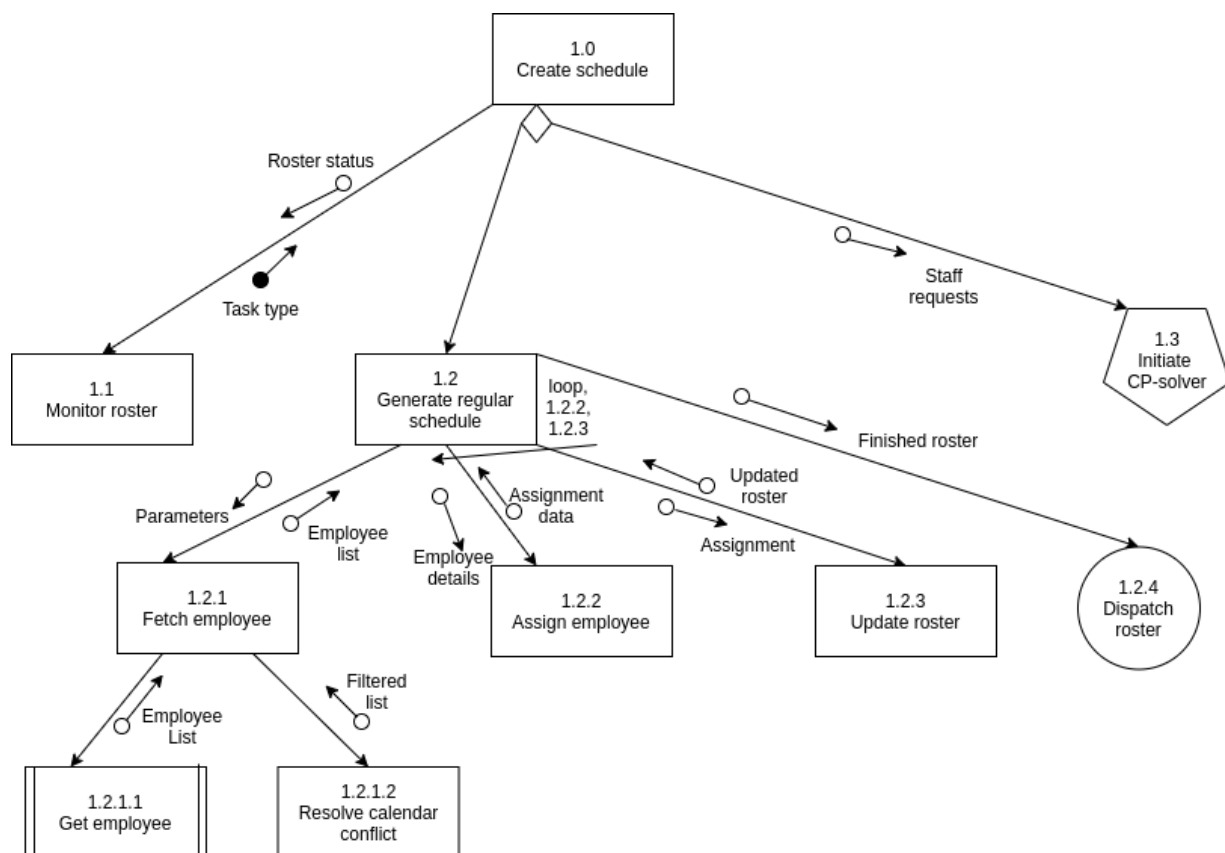


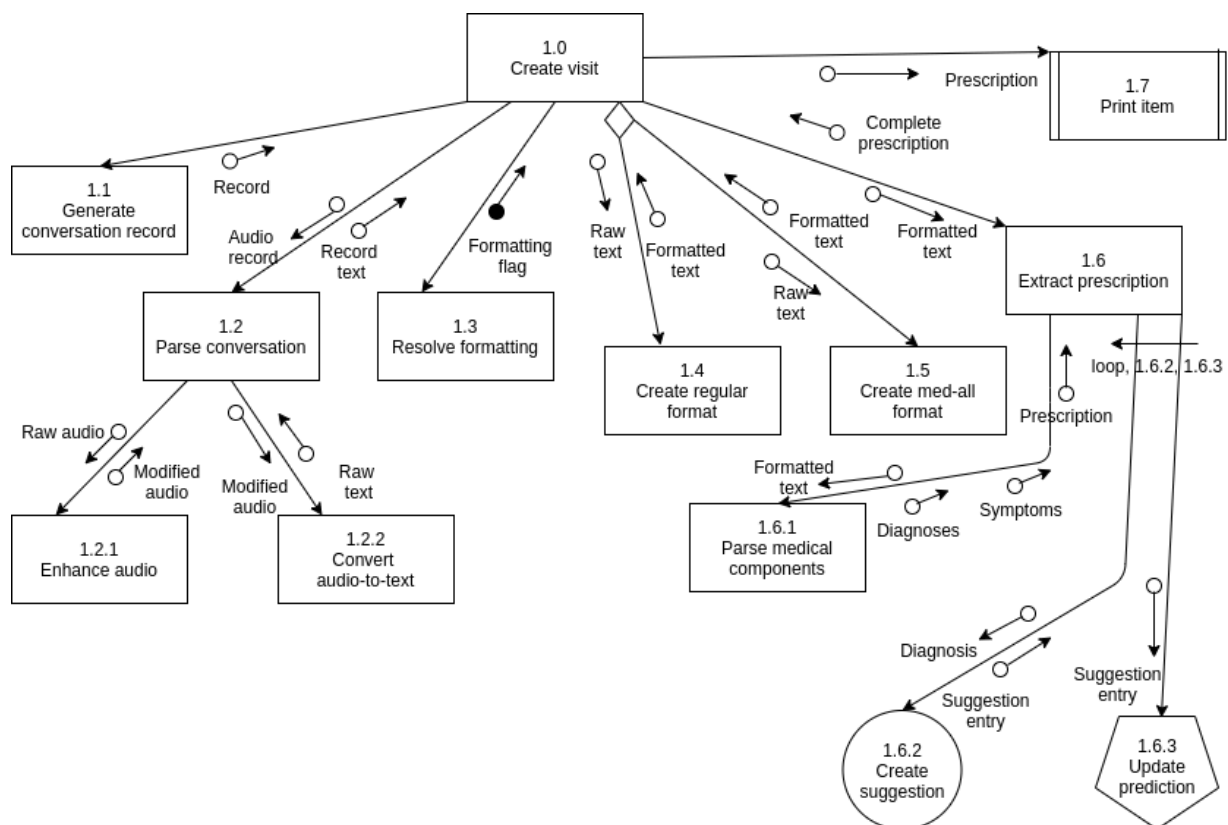
1. BRACU-Crawler is a pilot project designed to crawl BD sites only. First, Initiate crawler module provides current status of URL queue to the Schedule policy module and receives policy map and crawling threshold number. Until a threshold reached flag is received from the Fetch URL module, Crawler receives a link from Fetch URL module. It then retrieves a new page by providing page link to the Fetch site page module. This page is then sent to the Extract URL module and it returns a set of newly extracted links to the Crawler. In order to do that, Extract URL module first generates raw links using Parse raw links module and sends those to the Filter Valid links module. Filter valid links module returns only the URLs that Crawler should crawl in the future. After receiving the filtered links, Extract URL module formats the links using a library module called Format link and finally sends these filtered-formatted links to the Crawler. Based on the policy map, Crawler can either add the links to the Update URL queue module which is an off-page connector or dispatch to the Reschedule policy module which is an on-page connector.



2. BRACU-Scheduler is a program tasked with staff scheduling and weekly roster creation. Create schedule module first receives a task type flag from the Monitor roster module by providing the current roster status information. Based on the flag, Create schedule can either initiate a regular scheduling or a CP-solver by taking into account the staff requests. Generate regular schedule module first retrieves the employees on duty for next week from the Fetch employee module. In order to generate this employee list, Fetch employee module uses Get employee module which is a library module and Resolve calendar conflict module in the order they are mentioned and they return employee list and filtered employee list based on vacation notice, respectively. Until the final roster is created for all employees, Generate regular schedule calls upon the Assign Employee module with details of each employee and receives an assignment data. It then sends this data to Update roster module and receives updated roster. Finally, finished roster is passed onto the Dispatch roster module which is an on-page connector. On the other hand, Create schedule uses a Initiate CP-solver module to generate schedules with staff requests which is an off-page connector.



3. Modern-EHR is a healthcare program that generates patients' visits and prescriptions based on conversation recording. Create visit module starts by calling upon the Generate conversation record module to prepare a doctor-patient conversation record. It is then sent to a Parse conversation module which returns a text description of doctor-patient conversation to Create visit module. In order to generate text, Parse conversation module first sends raw audio data to Enhance audio module and receives a noise-reduced modified version. This is then passed onto Convert audio-to-text module which returns the raw text data. After receiving this text data, Create visit module receives a formatting flag from Resolve formatting module and based on this decides whether to use Create regular format or Create med-all format module. Create regular format returns a simply formatted text file using a generic formatter, whereas Create med-all format returns a specially formatted file using standard healthcare templates. This text file is passed onto the Extract prescription module. First it receives the extracted diagnoses, symptoms and prescription information from Parse medical components module. Next for each diagnosis, Create suggestion module which is an on-page connector is called to create a suggestion entry and then Update prediction module which is an off-page connector is called to update the prediction model for given suggestion entry. Finally Extract prescription module returns the complete prescription to Create visit module. Create visit module uses Print item library module to print the given prescription.



4. “HomeStar” is a home automation system to manage, operate and maintain household devices. Upon receiving the command “Prepare workspace”, Prepare HomeStar module is initiated. First, Generate parameters module is given a day/night status and it returns the parameter details for the status as well as an optimization control flag for system optimization to be carried out later. Prepare HomeStar sends the parameter details to Prepare batch module and receives a list of activated devices in return. In order to prepare all connected target devices, Prepare batch module fetches a list of target devices from Get devices module by providing the parameters. For each device, Prepare batch module first calls upon Get current status library module by providing the device id and in return receives whether the device is currently activated or not. If the device is not active, Activate device module is triggered using device id and parameter mapping. Otherwise Update device state module is triggered with similar information and the target device is updated with given parameters. After each device is activated/updated, Generate activation entry module creates formatted specification and returns an entry to Prepare batch. After all target devices are activated/updated, Prepare batch module sends the activated device list to Prepare HomeStar. Finally Prepare HomeStar initiates the Optimize system module which is an off-page connector and Initiate emergency alert module which is an on-page connector.

